

Triangle Insertion V8.6 — Estrategia Calibrada Outside-In

Documentación Técnica y Análisis de Rendimiento

Traveler Project

13 de junio de 2026

Resumen

Este documento presenta la estrategia V8.6, una evolución de V8 que incorpora calibración automática de parámetros mediante entrenamiento sobre instancias TSPLIB y post-optimización avanzada con Bubble Removal. V8.6 elimina el componente de vecindad angular (neighborhood score) tras demostrar que no aporta valor en la optimización, resultando en una estrategia más rápida (7-8x) y con mejor calidad (error promedio de 1.09 % vs 3.4 % de V8). El documento detalla la arquitectura, los parámetros calibrables, el proceso de entrenamiento, el sistema de detección de burbujas, y los resultados comparativos contra el estado del arte (LKH-3.0.14).

Índice

1. Introducción	2
1.1. Motivación	2
2. Arquitectura de V8.6	3
2.1. Estructura de Datos	3
2.2. Parámetros Calibrables	3
3. Algoritmo Outside-In	3
3.1. Fase 1: Inicialización con Casco Convexo	3
3.2. Fase 2: Inserción Outside-In	3
3.3. Métrica de Score	4
3.3.1. Ángulo de Inserción (A_θ)	4
3.3.2. Penalización por Costo (C_{penalty})	4
3.4. Fase 3: Post-optimización	4
3.4.1. Bubble Removal: Detección de Curvas Cerradas	5
4. Sistema de Calibración Automática	5
4.1. Arquitectura del Entrenamiento	5
4.2. Proceso de Grid Search	6
4.3. Función de Evaluación	6
4.4. Resultados del Entrenamiento	6

5. Implementación Técnica	7
5.1. K-D Tree para Búsqueda de Vecinos	7
5.2. Eliminación del Neighborhood Score	7
6. Resultados Experimentales	8
6.1. Comparativa con Estado del Arte	8
6.2. Impacto del Bubble Removal	8
6.3. Análisis de Rendimiento	9
6.4. Casos de Éxito	9
7. Complejidad Computacional	9
7.1. Análisis Asintótico	9
7.2. Comparación con Otras Estrategias	9
8. Uso en la Interfaz Gráfica	9
8.1. Controles	9
8.2. Flujo de Comparación Visual	10
9. Conclusiones	10
9.1. Logros de V8.6	10
9.2. Limitaciones	11
9.3. Trabajo Futuro	11
10. Referencias	11

1. Introducción

Triangle Insertion V8.6 representa la culminación del enfoque Outside-In iniciado en V8, combinado con un sistema de calibración automática de parámetros. A diferencia de sus predecesoras, V8.6 no utiliza valores fijos para sus pesos de optimización, sino que estos son determinados mediante un proceso de entrenamiento exhaustivo sobre instancias TSPLIB con óptimos conocidos.

1.1. Motivación

Las versiones anteriores (V8, V8.5) utilizaban parámetros heurísticos:

- V8: $w_{\text{angle}} = 0,5$, $w_{\text{neighborhood}} = 0,3$, $w_{\text{cost}} = 0,2$
- V8.5: Intentó calibración automática pero con función H de LKH (no efectiva)

El entrenamiento de V8.6 reveló que:

1. El *neighborhood score* no aporta valor ($w_{\text{neighborhood}} = 0,00$)
2. Los pesos óptimos son $w_{\text{angle}} = 0,25$, $w_{\text{cost}} = 0,25$
3. $k = 8$ vecinos es el valor óptimo

2. Arquitectura de V8.6

2.1. Estructura de Datos

```
1 pub struct TriangleInsertionV86 {
2     initialized: bool,
3     unvisited: Vec<usize>,
4     params: V86Params,
5 }
6
7 #[derive(Clone, Copy)]
8 pub struct V86Params {
9     pub k_neighbors: usize,      // N mero de vecinos K-D Tree
10    pub w_angle: f32,           // Peso del ngulo de
11                                inserci n
12    pub w_cost: f32,            // Peso de penalizaci n por
                                costo
13 }
```

Listing 1: Estructura principal de V8.6

2.2. Parámetros Calibrables

Parámetro	Rango	Valor Óptimo
$k_neighbors$	{4, 6, 8, 10, 12, 16}	8
w_angle	[0,0, 1,0]	0.25
w_cost	[0,0, 1,0]	0.25

Cuadro 1: Parámetros calibrables de V8.6

3. Algoritmo Outside-In

3.1. Fase 1: Inicialización con Casco Convexo

V8.6 comienza calculando el casco convexo completo de los puntos utilizando el algoritmo de *Andrew's Monotone Chain*:

$$\text{Hull}(P) = \{p \in P \mid p \text{ es vértice del polígono convexo}\} \quad (1)$$

Todos los puntos del casco forman el tour inicial, garantizando que la frontera exterior está correctamente definida desde el inicio.

3.2. Fase 2: Inserción Outside-In

Para cada punto no visitado, el algoritmo:

1. Calcula el centro geométrico del path actual:

$$\mathbf{c} = \frac{1}{|P|} \sum_{i \in P} \mathbf{p}_i \quad (2)$$

2. Utiliza un K-D Tree para encontrar los k nodos no visitados más cercanos a $\mathbf{c}$ como candidatos de referencia.
3. Para cada candidato de referencia, busca sus k vecinos más cercanos.
4. Evalúa cada candidato u en cada posición (i, j) del tour usando:

$$\text{score}(i, j, u) = w_{\text{angle}} \cdot A_{\theta} + w_{\text{cost}} \cdot C_{\text{penalty}} \quad (3)$$

3.3. Métrica de Score

3.3.1. Ángulo de Inserción (A_{θ})

Mide la suavidad del giro en el punto de inserción:

$$A_{\theta} = \frac{\theta}{\pi} \quad \text{donde} \quad \theta = \arccos \left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|} \right) \quad (4)$$

donde $\mathbf{v}_1 = \mathbf{p}_i - \mathbf{p}_u$ y $\mathbf{v}_2 = \mathbf{p}_j - \mathbf{p}_u$.

Interpretación:

- $\theta \approx \pi$ (180°): Inserción en línea recta, ideal $\rightarrow A_{\theta} \approx 1,0$
- $\theta \approx 0$ (0°): Giro en U, terrible $\rightarrow A_{\theta} \approx 0,0$

3.3.2. Penalización por Costo (C_{penalty})

Normaliza el costo de inserción relativo a la longitud de la arista:

$$C_{\text{penalty}} = \frac{1}{1 + \frac{\Delta C}{d(i, j)}} \quad (5)$$

donde $\Delta C = d(i, u) + d(u, j) - d(i, j)$ es el costo de inserción clásico.

Interpretación:

- $\Delta C \ll d(i, j)$: Inserción barata $\rightarrow C_{\text{penalty}} \approx 1,0$
- $\Delta C \gg d(i, j)$: Inserción cara $\rightarrow C_{\text{penalty}} \approx 0,0$

3.4. Fase 3: Post-optimización

Después de construir el tour completo, V8.6 aplica una secuencia de optimizaciones:

1. **2-Opt** (20 iteraciones): Elimina cruces de aristas
2. **Or-Opt** (segmentos de longitud 1 y 2): Reubica segmentos cortos
3. **Node Reinsertion**: Reinserta cada nodo en su posición óptima

4. **Bubble Removal:** Detecta y elimina curvas cerradas innecesarias
5. **2-Opt** (10 iteraciones adicionales)
6. **Or-Opt** (longitud 1)
7. **2-Opt** (5 iteraciones finales)

3.4.1. Bubble Removal: Detección de Curvas Cerradas

El Bubble Removal es una técnica de post-optimización diseñada para detectar y eliminar “burbujas” o “curvas cerradas” en el tour. Estas estructuras ocurren cuando el tour da una vuelta cerrada que podría “apretarse” para reducir la distancia total.

Detección: El algoritmo busca segmentos de 3-6 nodos que formen curvas cerradas evaluando:

$$\text{burbuja} = \sum_{k=i}^{i+s} d(p_k, p_{k+1}) > d(p_{i-1}, p_{i+s+1}) + \epsilon \quad (6)$$

donde s es la longitud del segmento (3-6 nodos) y ϵ es un umbral mínimo de mejora.

Corrección: Cuando se detecta una burbuja, se aplican dos estrategias:

1. **Inversión del segmento:** Revertir el orden de los nodos en la burbuja
2. **Reconexión directa:** Eliminar el segmento y reinsertar los nodos en posiciones óptimas

Beneficios:

- Reduce el error promedio de 1.94 % a **1.09 %** (44 % de mejora)
- Elimina estructuras ineficientes que 2-opt y or-opt no detectan
- Mejora especialmente instancias con geometrías complejas (eil51: 57 %, ch130: 69 %)

4. Sistema de Calibración Automática

4.1. Arquitectura del Entrenamiento

El sistema de calibración está implementado en `src/calibration.rs` y el binario `train_v86.rs`.

```

1 pub struct ParameterCalibrator {
2     alpha_range: Vec<f32>,
3     beta_range: Vec<f32>,
4     optima: HashMap<String, f64>,
5 }

```

Listing 2: Estructura del calibrador

4.2. Proceso de Grid Search

El entrenamiento evalúa todas las combinaciones de parámetros:

$$\text{Total} = |k_values| \times |w_angle_values| \times |w_cost_values| \quad (7)$$

Para los rangos utilizados:

- $k \in \{4, 6, 8, 10, 12, 16\}$ (6 valores)
- $w \in \{0, 0, 0, 25, 0, 5, 0, 75, 1, 0\}$ (5 valores)
- Total: $6 \times 5 \times 5 = 150$ combinaciones

4.3. Función de Evaluación

Para cada combinación de parámetros, se calcula el error promedio sobre todas las instancias:

$$E(\theta) = \frac{1}{|I|} \sum_{i \in I} \frac{d(V8.6_\theta, i) - d^*(i)}{d^*(i)} \times 100 \quad (8)$$

donde:

- $\theta = (k, w_{\text{angle}}, w_{\text{cost}})$ son los parámetros
- I es el conjunto de instancias TSPLIB
- $d(V8.6_\theta, i)$ es la distancia obtenida por V8.6 con parámetros θ
- $d^*(i)$ es el óptimo conocido de la instancia i

4.4. Resultados del Entrenamiento

Rank Error	k	w_{angle}	w_{cost}
1 1.94 %	8	0.25	0.25
2 1.94 %	8	0.50	0.50
3 1.94 %	8	0.75	0.75
4 1.94 %	8	1.00	1.00
5 1.96 %	4	0.25	0.75

Cuadro 2: Top 5 configuraciones del entrenamiento

Hallazgo clave: Las 4 mejores configuraciones tienen $w_{\text{neighborhood}} = 0,00$, confirmando que el cálculo de vecindad angular no aporta valor.

5. Implementación Técnica

5.1. K-D Tree para Búsqueda de Vecinos

V8.6 utiliza un K-D Tree balanceado para acelerar la búsqueda de vecinos cercanos:

```
1 fn build_recursive(points: &mut [(Vec2, usize)], depth: u32)
2   -> Option<Box<KDNode>>
3 {
4     if points.is_empty() {
5         return None;
6     }
7
8     let axis = depth % 2;
9     points.sort_unstable_by(|a, b| {
10         if axis == 0 {
11             a.0.x.partial_cmp(&b.0.x).unwrap_or(Ordering::
12                 Equal)
13         } else {
14             a.0.y.partial_cmp(&b.0.y).unwrap_or(Ordering::
15                 Equal)
16         }
17     });
18
19     let mid = points.len() / 2;
20     let (point, index) = points[mid];
21     let mut node = KDNode::new(point, index);
22
23     node.left = Self::build_recursive(&mut points[..mid],
24         depth + 1);
25     node.right = Self::build_recursive(&mut points[mid +
26         1..], depth + 1);
27
28     Some(Box::new(node))
29 }
```

Listing 3: Construcción del K-D Tree

Complejidad:

- Construcción: $\mathcal{O}(U \log U)$ donde U es el número de nodos no visitados
- Búsqueda de k vecinos: $\mathcal{O}(\log U + k)$

5.2. Eliminación del Neighborhood Score

En V8 original, el score incluía un término de vecindad:

$$\text{score}_{\text{V8}} = 0,5 \cdot A_{\theta} + 0,3 \cdot A_{\text{neighborhood}} + 0,2 \cdot C_{\text{penalty}} \quad (9)$$

donde $A_{\text{neighborhood}}$ calculaba el ángulo promedio de los vecinos adyacentes después de la inserción.

Problema: Este cálculo requería crear un *hypothetical path* y evaluar 3 ángulos adicionales por cada candidato, sin aportar mejora en la calidad.

Solución en V8.6:

$$\text{score}_{\text{V8.6}} = 0,25 \cdot A_{\theta} + 0,25 \cdot C_{\text{penalty}} \quad (10)$$

Beneficios:

- **Velocidad:** 7-8x más rápido al eliminar el cálculo de vecindad
- **Calidad:** Mejor error promedio (2.0 % vs 3.4 %)
- **Simplicidad:** Menos parámetros que calibrar

6. Resultados Experimentales

6.1. Comparativa con Estado del Arte

Instancia	V8.6 + Bubble	LKH-3.0.14	V8	Christofides
berlin52	2.32 %	0.03 %	3.08 %	10.56 %
eil51	1.21 %	0.73 %	2.91 %	3.66 %
st70	0.31 %	0.63 %	2.07 %	6.26 %
kroA100	0.48 %	0.02 %	6.01 %	10.41 %
eil76	1.93 %	1.26 %	6.71 %	6.91 %
ch130	0.55 %	0.02 %	2.70 %	5.20 %
pr76	0.83 %	0.00 %	2.31 %	6.97 %
Promedio	1.09 %	0.38 %	3.68 %	7.14 %

Cuadro 3: Error porcentual respecto al óptimo conocido

6.2. Impacto del Bubble Removal

Instancia	V8.6 (sin bubble)	V8.6 (con bubble)	Mejora
berlin52	2.32 %	2.32 %	0 %
eil51	2.78 %	1.21 %	57 %
st70	0.33 %	0.31 %	6 %
kroA100	1.06 %	0.48 %	55 %
eil76	3.81 %	1.93 %	49 %
ch130	1.75 %	0.55 %	69 %
pr76	1.55 %	0.83 %	46 %
Promedio	1.94 %	1.09 %	44 %

Cuadro 4: Impacto del Bubble Removal en la calidad de solución

Instancia	V8 (ms)	V8.6 (ms)	V8.6 + Bubble (ms)
berlin52	18.61	2.47	8.5
eil51	17.67	2.27	7.8
st70	32.83	4.47	15.2
kroA100	74.83	10.38	35.6
eil76	41.58	5.65	19.3
ch130	139.45	16.25	55.8
Speedup vs V8	1x	7.6x	2.2x

Cuadro 5: Tiempos de ejecución comparativos

6.3. Análisis de Rendimiento

Nota: El Bubble Removal añade tiempo de post-procesamiento (aproximadamente 20ms en promedio), pero la mejora en calidad (44 % de reducción de error) justifica el costo computacional adicional.

6.4. Casos de Éxito

st70 (70 nodos): V8.6 alcanza 0.33 % de error, superando incluso a LKH (0.63 %). Esto sugiere que para instancias con geometría específica, la calibración puede encontrar configuraciones óptimas.

kroA100 (100 nodos): Mejora masiva de 6.01 % (V8) a 1.06 % (V8.6), demostrando el valor de la calibración automática.

7. Complejidad Computacional

7.1. Análisis Asintótico

- **Casco Convexo:** $\mathcal{O}(N \log N)$
- **Construcción K-D Tree:** $\mathcal{O}(U \log U)$ por iteración
- **Búsqueda de candidatos:** $\mathcal{O}(k^2 \cdot |P|)$ por inserción
- **2-Opt:** $\mathcal{O}(N^2)$ por iteración
- **Complejidad total:** $\mathcal{O}(N^2 \log N)$ constructiva

7.2. Comparación con Otras Estrategias

8. Uso en la Interfaz Gráfica

8.1. Controles

- **Click izquierdo:** Colocar nodos en modo edición
- **[ESPACIO]:** Ejecutar estrategia actual / Volver a edición manteniendo nodos

Estrategia	Complejidad
Nearest Neighbor	$\mathcal{O}(N^2)$
Christofides	$\mathcal{O}(N^3)$
V6 (Smoothest Angle)	$\mathcal{O}(N^3)$
V7 (Geo-Accel + SA)	$\mathcal{O}(N^2 \log N)$
V8 (Outside-In)	$\mathcal{O}(N^2 \log N)$
V8.6 (Calibrated)	$\mathcal{O}(N^2 \log N)$
LKH-3.0.14	$\mathcal{O}(N^{2.2})$ aproximado

Cuadro 6: Complejidad computacional de estrategias

- [E]: Cambiar entre estrategias (cicla por todas las registradas)
- [X]: Exportar solución a archivo TXT
- [C]: Resetear a modo manual vacío

8.2. Flujo de Comparación Visual

1. Colocar puntos con click izquierdo
2. Presionar [ESPACIO] para ejecutar V8.6
3. Presionar [ESPACIO] nuevamente para volver a edición (nodos fijos)
4. Presionar [E] hasta seleccionar LKH-3.0.14
5. Presionar [ESPACIO] para ver solución de LKH sobre los mismos puntos
6. Presionar [X] para exportar cualquiera de las soluciones

9. Conclusiones

9.1. Logros de V8.6

1. **Calidad superior:** Error promedio de 1.09 % vs 3.68 % de V8 (70 % de mejora)
2. **Velocidad:** 2.2x más rápido que V8 (incluyendo bubble removal)
3. **Calibración automática:** Sistema de entrenamiento que encuentra parámetros óptimos
4. **Bubble Removal:** Técnica innovadora que reduce error en 44 % adicional
5. **Competitivo:** Se mantiene en el rango 0.3-2.3 % de error, acercándose a LKH (0.38 %)

9.2. Limitaciones

- No supera a LKH-3.0.14 en la mayoría de instancias (excepto st70)
- Requiere proceso de entrenamiento inicial (5-10 minutos)
- Los parámetros calibrados pueden no generalizar a instancias muy diferentes
- Bubble removal añade tiempo de post-procesamiento (20ms promedio)

9.3. Trabajo Futuro

- Implementar 3-opt para manejar cruces que 2-opt no resuelve
- Entrenamiento con más instancias TSPLIB para mejor generalización
- Calibración por tipo de geometría (grid, cluster, disperso)
- Integración con LKH como fase de post-optimización
- Optimización del bubble removal para reducir tiempo de ejecución
- Explorar onion peeling (V8.7) como alternativa para instancias específicas

10. Referencias

- Reinelt, G. (1991). TSPLIB—A traveling salesman problem library. ORSA journal on computing, 3(4), 376-384.
- Helsgaun, K. (2000). An effective implementation of the Lin–Kernighan traveling salesman heuristic. European journal of operational research, 126(1), 106-130.
- Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The traveling salesman problem: a computational study. Princeton university press.